GATE
CS & DA

Artificial Intelligence

Informed Search

Lecture No. 5

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

Topic — AO^*

Topic — A^*, GBFS

Topic — Open/Close list

Topic — node generated / node Inspected

Topic

Topics to be Covered

- Topic AO*
- Topic IDA*
- Topic
- Topic
- Topic

make yourself
Better
$Be +ve

Be the change that you wish to see in the world.

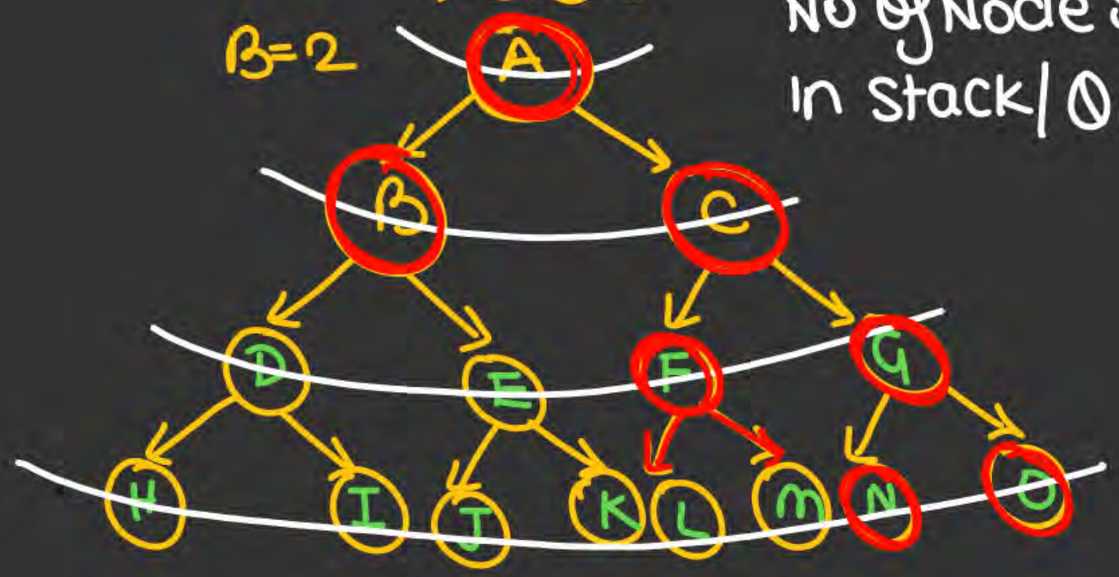
[redacted]

BRIAN TRACY
INTERNATIONAL

Queue/Stack
Space Complexity
decided by
No of Node stored
in Stack/Queue

BFS

DFS



~~A~~ ~~B~~ ~~C~~ ~~D~~ ~~E~~ ~~F~~ ~~G~~ HIJKLMNO

(2^3) 8

~~A~~ ~~B~~ ~~C~~ ~~F~~ ~~G~~ ~~N~~ ~~O~~

$\Rightarrow O(b \times d)$

Space Complexity

$\Downarrow$

$O(b \cdot d)$ ✓

Time Complexity

$\downarrow$

we have to go to all nodes if heuristic are not good

$\rightarrow O(b^d)$

AO* vs A*

. Complexity

A* → Space ⇒ → O(b^d)
A* → Time ⇒ → O(b^d)

- *AO Algorithm:**
  - **Complexity**: More complex due to the need to handle both AND and OR nodes, and the need to maintain multiple paths that might need to be explored simultaneously.
  - **Space and Time**: The complexity depends on the structure of the AND-OR graph and the quality of the heuristic.
- *A Algorithm:**
  - **Complexity**: Generally less complex than AO* as it deals with a simpler graph structure.
  - **Space and Time**: Exponential in the worst case, as A* may explore many nodes, but it is highly efficient with a good heuristic.

Time Complexity of AO*

Time Complexity

The time complexity of the AO* algorithm is challenging to define precisely because it heavily depends on several factors:

1. **Size of the Graph:** The number of nodes (N) and edges (E) in the And-Or graph.
2. **Branching Factor:** The average number of successors for each node.
3. **Heuristic Function:** The quality of the heuristic function used to guide the search.

In general, the time complexity can be expressed as:

$$O(b^d)$$

where:

- b is the branching factor (average number of child nodes),
- d is the depth of the graph or the solution path.

Space Complexity of AO*

Space Complexity

The space complexity of the AO* algorithm is similarly dependent on the structure of the graph and the efficiency of the heuristic:

1. **Space for Nodes:** The algorithm needs to store all the nodes that are part of the search tree, which includes the nodes on the current solution path, the fringe nodes (nodes yet to be expanded), and the nodes that have already been expanded.
2. **Space for Heuristics and Backtracking:** Additional space is required for storing heuristic values and managing the backtracking process when the solution needs to be refined.

The space complexity is generally given as:

$O(b \cdot d)$

where:

- b is the branching factor,
- d is the depth of the search tree or solution path.

This reflects that, in the worst case, the algorithm needs to store all the nodes at each level of the tree up to the maximum depth m.

IDA*

⇒ Iterative deepening A*

→ A* ⇒ Space $O(b^d)$
Time $O(b^d)$

→ IDA* ⇒ Space $O(b \cdot d)$
Time $O(b^d)$

IDDFS
depth = 0, 1, 2, 3 - -

IDDFS
Space Comp. $O(b \cdot d)$
Time Comp $O(b^d)$

⇒ Iterative A*
Iteration Run on the f score

Algo:-

$f_{score} \Rightarrow g(n) + h(n)$

Start Node $\Rightarrow$ So h value of Start node $\Rightarrow f_{threshold}$

$f = g + h$

$f = 0 + h$

$f = h$

Iteration 1

we observe all the neighbours of the Current node, find f value of each neighbour

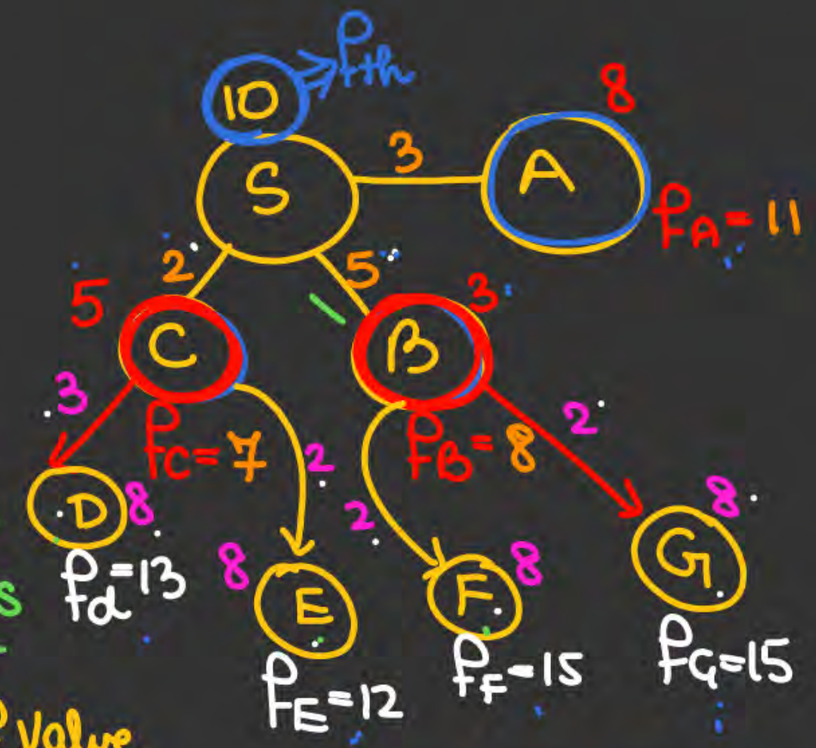
Now those neighbours will be visited which have $f_{score} \leq f_{threshold}$

If goal node reached ✓

If not $\Rightarrow f_{th} =$ min f of Pruned node

Iteration 1 Stop @ A, D, E, F, G

$\rightarrow$ Pruned node



So @ end of iteration 1 $f_{th} = 11$ we move to iteration 2.

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- The Iterative Deepening A* (IDA*) algorithm is an extension of the A* search algorithm designed to use less memory, making it more suitable for large search spaces where A* would require too much memory to store all nodes in the open and closed lists.
- IDA* combines the space efficiency of Iterative Deepening Depth-First Search (IDDFS) with the optimality of A*. Instead of maintaining a priority queue like A*, IDA* performs a series of depth-limited searches, where the depth limit is determined by a threshold that is increased iteratively. Each iteration explores paths that do not exceed the current threshold, and the threshold is based on the f-cost (sum of g-cost and h-cost).

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- The evaluation function in IDA* looks like this:

$$f(n) = g(n) + h(n)$$
$$f(n) = Actual\ cost + Estimated\ cost$$

- Where h is admissible.
- here,
- f(n) = Total cost evaluation function.
- g(n) = The actual cost from the initial node to the current node.
- h(n) = Heuristic estimated cost from the current node to the goal state. it is based on the approximation according to the problem characteristics.
- Threshold: In each iteration, IDA* sets a threshold based on the f-cost. It only explores nodes where f(n)≤f(n)≤ current threshold.

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- Algorithm Steps
- Initial Threshold:
- Set the initial threshold to f(start)=g(start)+h(start), where g(start)=0.
- Depth-Limited Search:
- Perform a depth-first search (DFS) where nodes are only expanded if their f-cost does not exceed the current threshold.
- Update Threshold:
- If the goal node is not found in the current iteration, set the threshold to the minimum f-cost of any node that exceeded the current threshold in the previous iteration.
- Repeat: Repeat the DFS with the new threshold until the goal node is found or no nodes remain to explore

Iterative Deepening A* algorithm (IDA*) – The complete Algorithm

- Step 1: Initialization
- Set the root node as the current node, and find the f-score.
- Sep 2: Set threshold
- Set the cost limit as a threshold for a node i.e the maximum f-score allowed for that node for further explorations.
- Step 3: Node Expansion
- Expand the current node to its children and find f-scores.
- Step 4: Pruning
- If for any node the f-score > threshold, prune that node because it's considered too expensive for that node. and store it in the visited node list.
- Step 5: Return Path
- If the Goal node is found then return the path from the start node Goal node.

Iterative Deepening A* algorithm (IDA*) – The complete Algorithm

- Step 6: Update the Threshold
- If the Goal node is not found then repeat from step 2 by changing the threshold with the minimum pruned value from the visited node list. And Continue it until you reach the goal node.

Iterative Deepening A* algorithm (IDA*) – The complete Algorithm

- Example
- In the below tree, the f score is written inside the nodes means the f score is already computed and the start node is 2 whereas the goal node is 15. the explored node is colored green color.
- so now we have to go to a given goal by using IDA* algorithm.

It 1: $f_{th}=2$, A B Pruned node

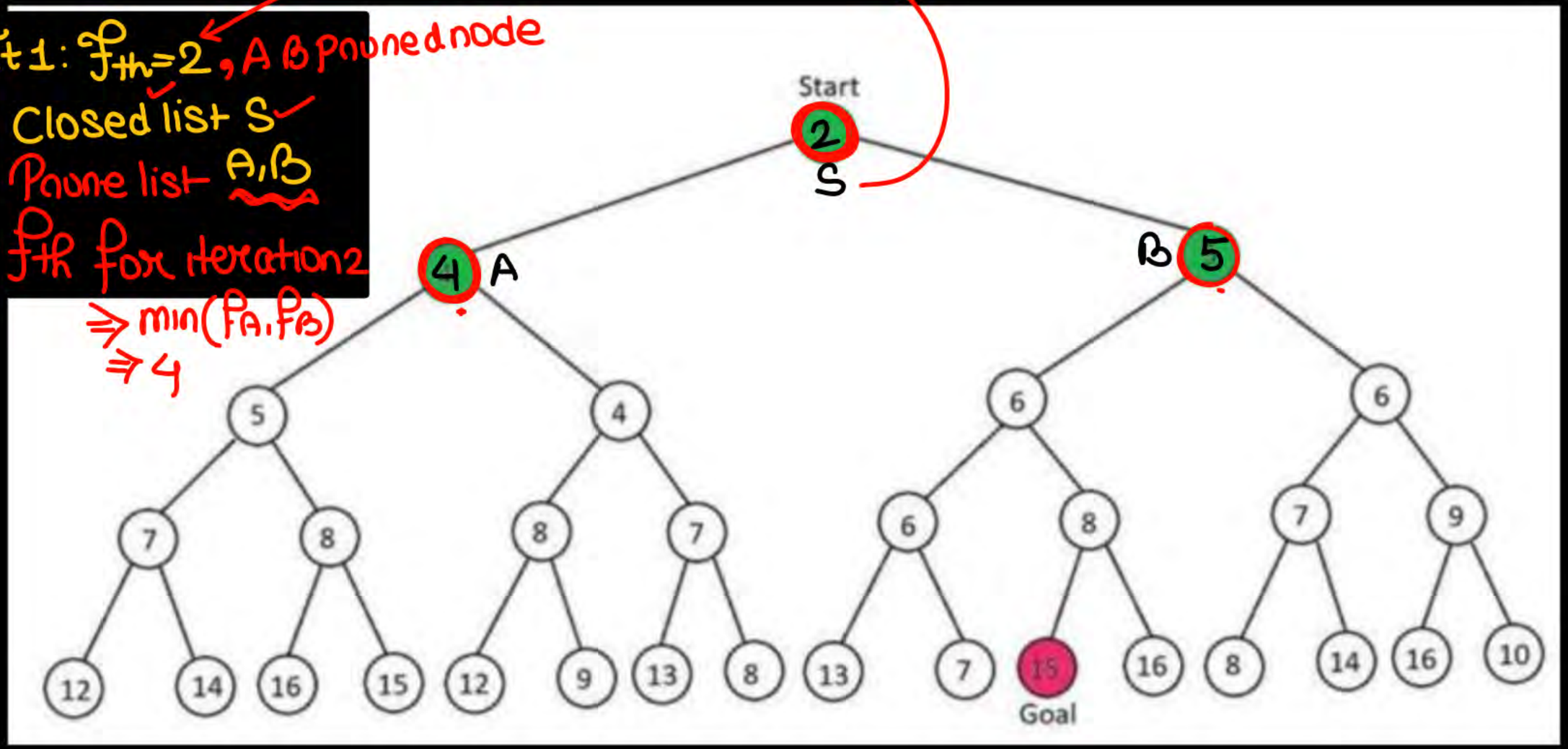
Closed list S

Prune list A, B

• f_{th} for iteration 2

$\Rightarrow \min(f_A, f_B)$

$\Rightarrow 4$



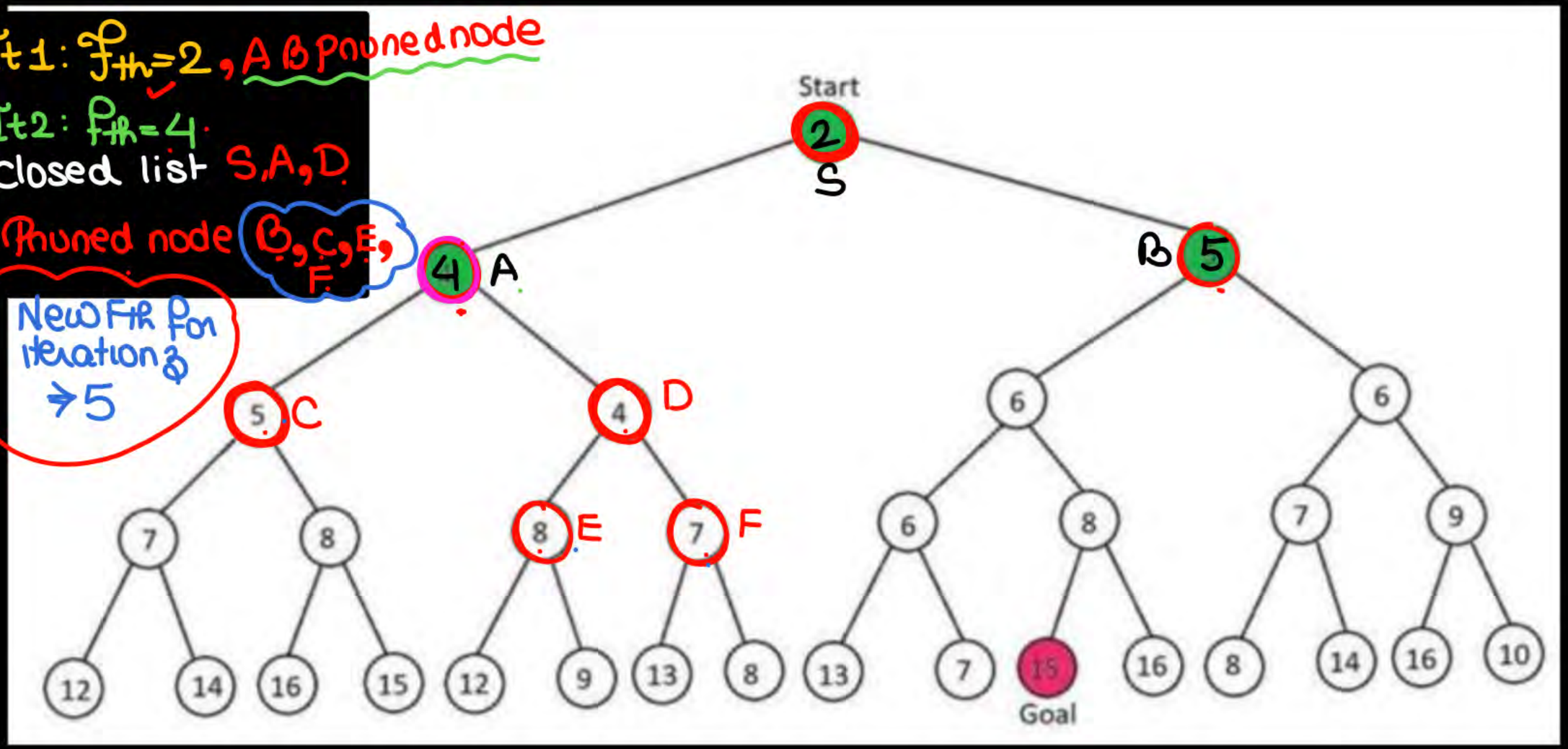
It1: $f_{th}=2$, A B Pruned node

→ It2: $f_{th}=4$.

Closed list S, A, D

Pruned node B, C, E, F

New f_{th} for iteration 3 → 5



It: 3

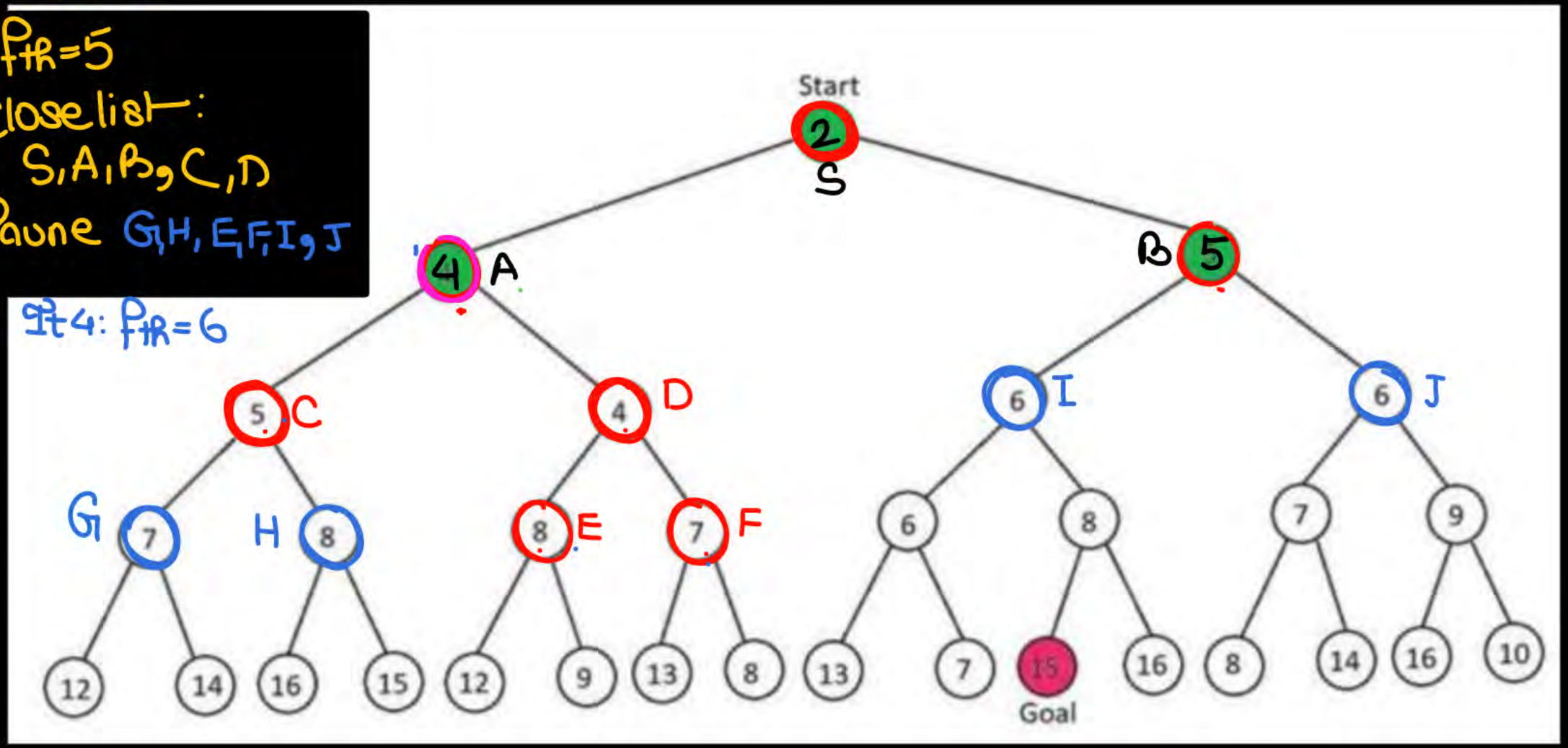
$f_{th}=5$

Closelist:

S, A, B, C, D

Pause G, H, E, F, I, J

It 4: $f_{th}=6$



It : 4

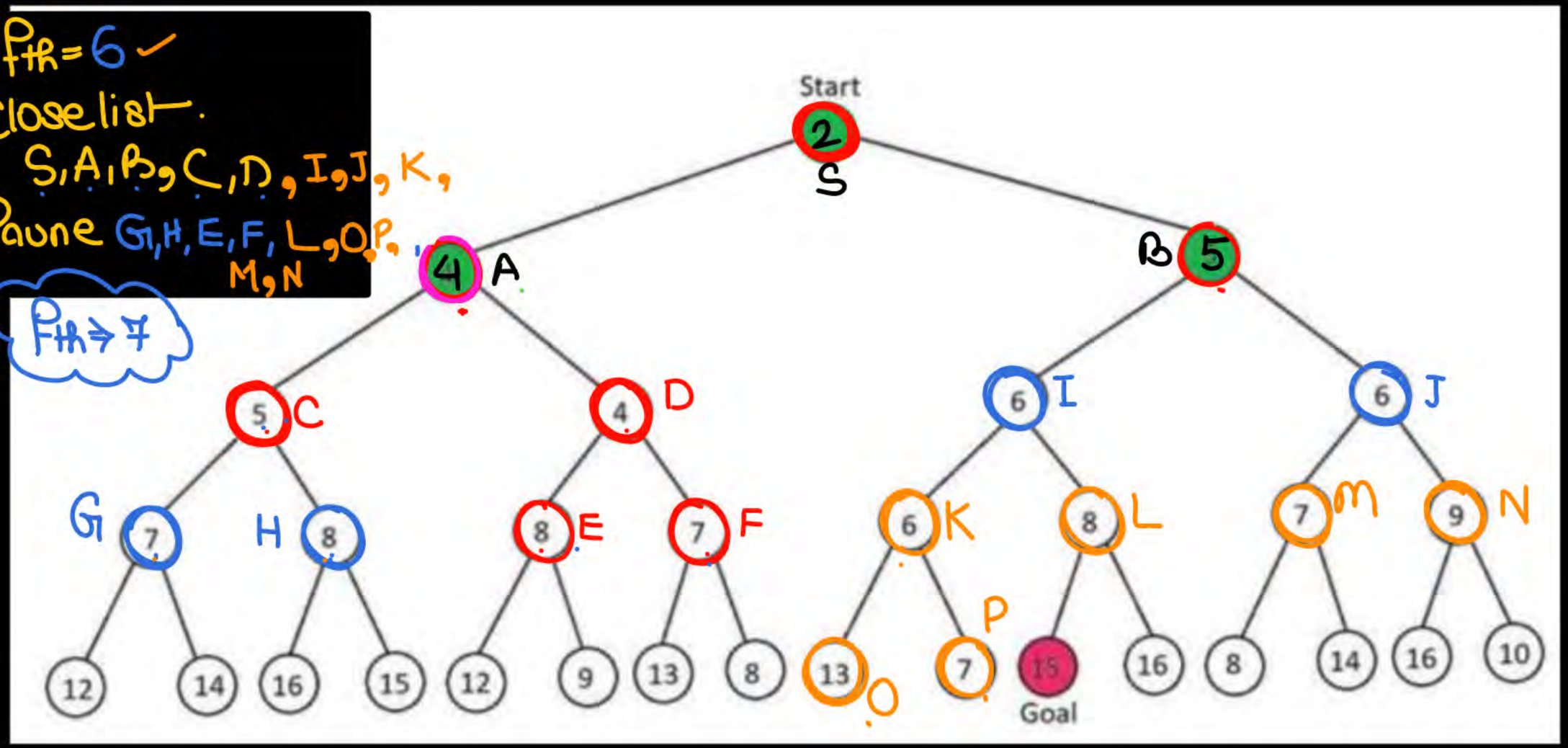
$f_{th} = 6$ ✓

Close list :

S, A, B, C, D, I, J, K,

Pause G, H, E, F, L, O, P, M, N

$f_{th} \Rightarrow 7$



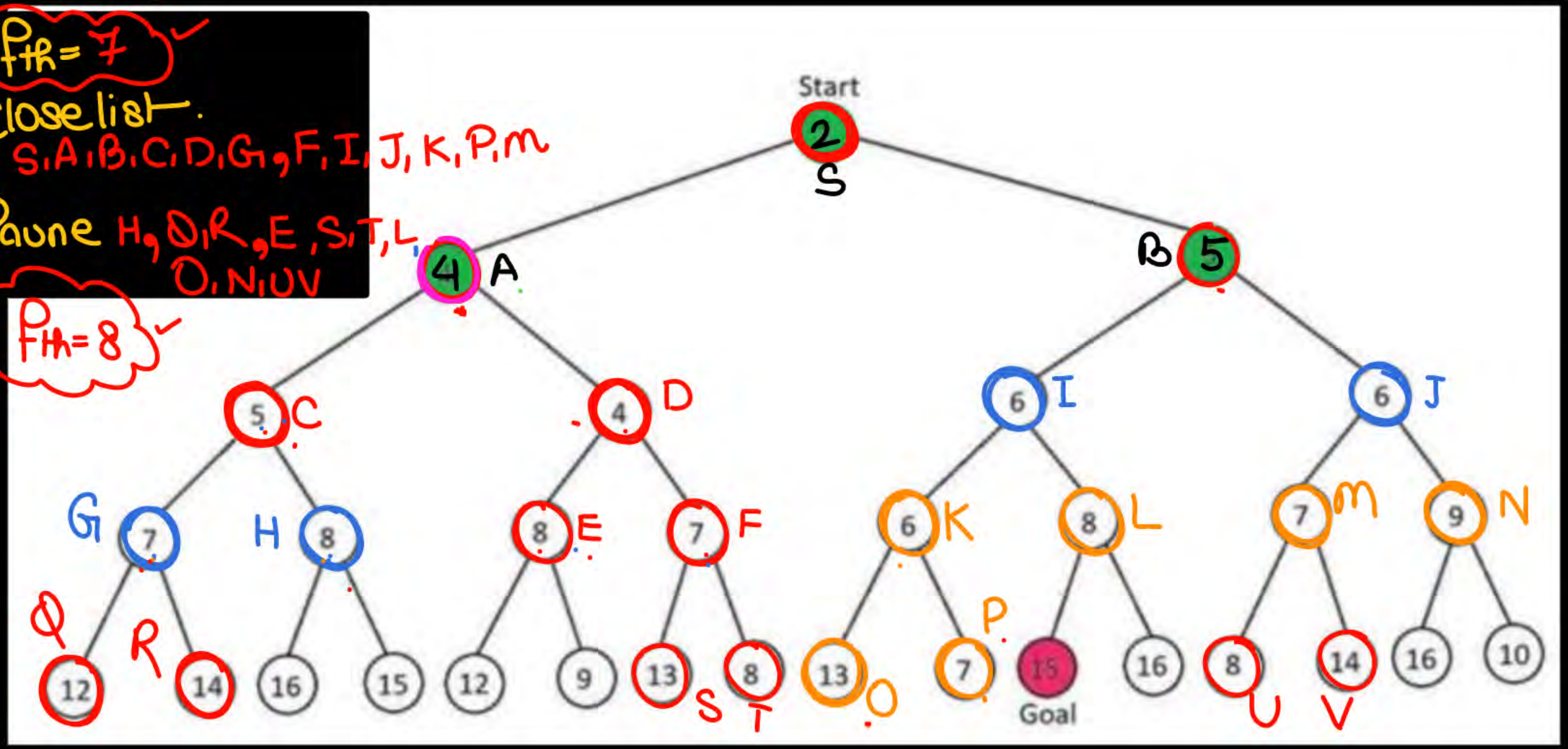
It : 5

$f_{th} = 7$

Close list :
S, A, B, C, D, G, F, I, J, K, P, M

Pause H, Q, R, E, S, T, L, O, N, U, V

$f_{th} = 8$



It: 6

$f_{th} = 8$

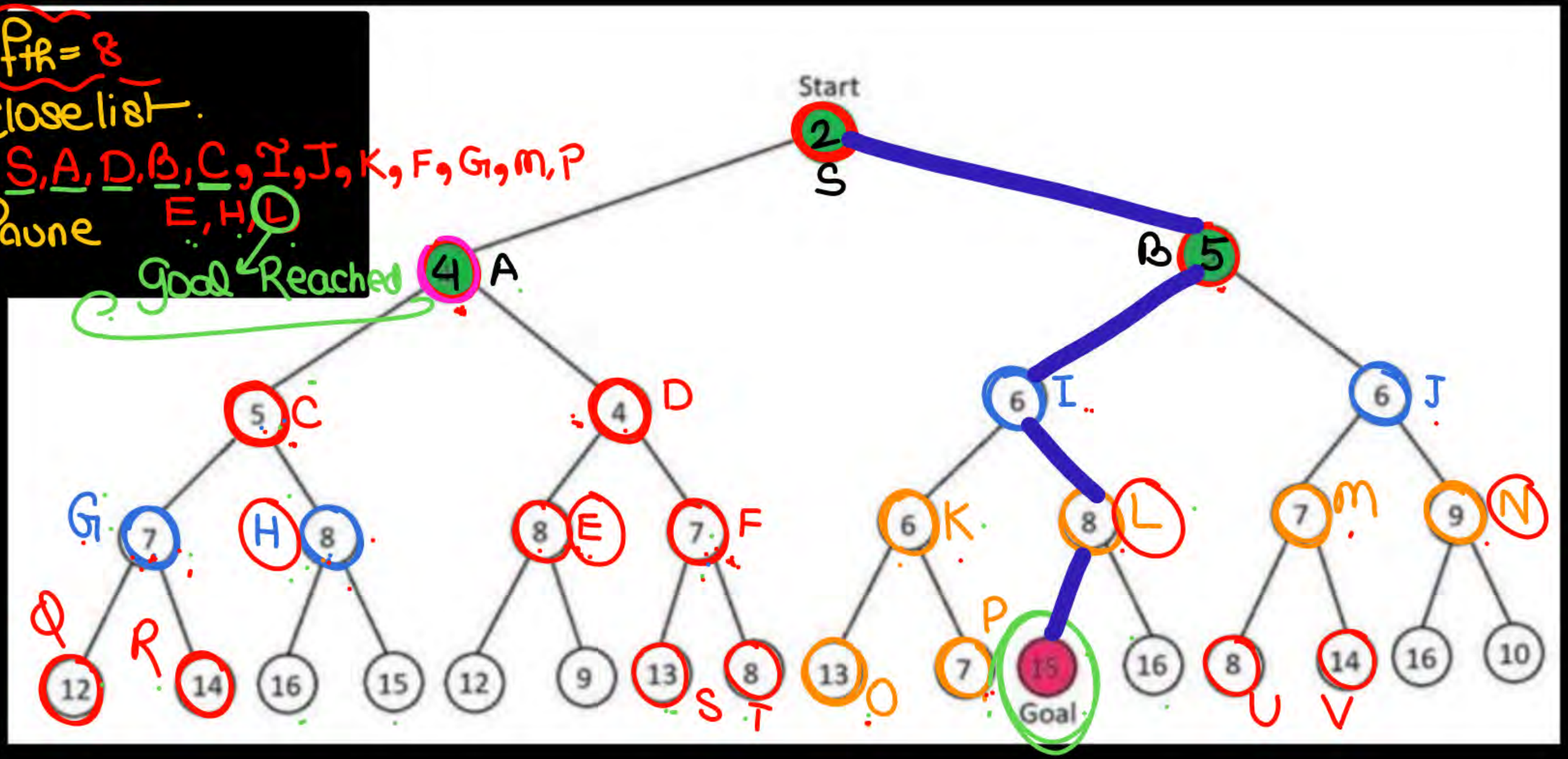
Close list.

S, A, D, B, C, I, J, K, F, G, M, P

Pause

E, H, L

Goal Reached



Consider a grid-based pathfinding problem where the agent needs to move from a start position S to a goal position G on a 5x5 grid. The cost to move from one cell to an adjacent cell is 1. Diagonal movements are allowed with a cost of $\sqrt{2}$. Next iℓ $f_{th} = 7.82$.

You are given the following heuristic values $h(x)$ based on the Manhattan distance:

It 1 ⇒ $f_{th} \Rightarrow 6$

$f_A = 1+8 = 9$ ✓

$f_C = 1+7 = 8$ ✓

$f_b = 5+\sqrt{2} \Rightarrow 6.414$

Prune

It 2 $f_{th} = 6.414$

S, b, g,

Prune a, c, d, f, h, e

	0	1	2	3	4
0	6	8 a	6 d	5	9
1	c 7	5 b	7 e	3	2
2	6 h	3 g	8 f	6	5
3	i 4	j 9	k 4	6	2
4	5	4	3	2	0

$f_a \Rightarrow 2\sqrt{2}+6 \rightarrow$ Prune $= 8.824$

$f_e \Rightarrow \sqrt{2}+1+7 \Rightarrow$ Prune ⇒

$f_f \Rightarrow 8+2\sqrt{2} \rightarrow$ Prune ⇒

$f_g \Rightarrow (1+\sqrt{2}+3) \leftarrow$ Visit

$f_h \Rightarrow 2\sqrt{2}+6 \rightarrow$ Prune ⇒ 8.824

$f_i \Rightarrow 2\sqrt{2}+1+4 = 7.82$

$f_j \Rightarrow 2+\sqrt{2}+9$

$f_k \Rightarrow 2\sqrt{2}+1+4 = 7.82$

Prune

The start position S is at $(0, 0)$ and the goal position G is at $(4, 4)$.

Using the IDA* algorithm, determine the first three threshold values for the iterative deepening search.

IDA

1st It → I Rest all Prune

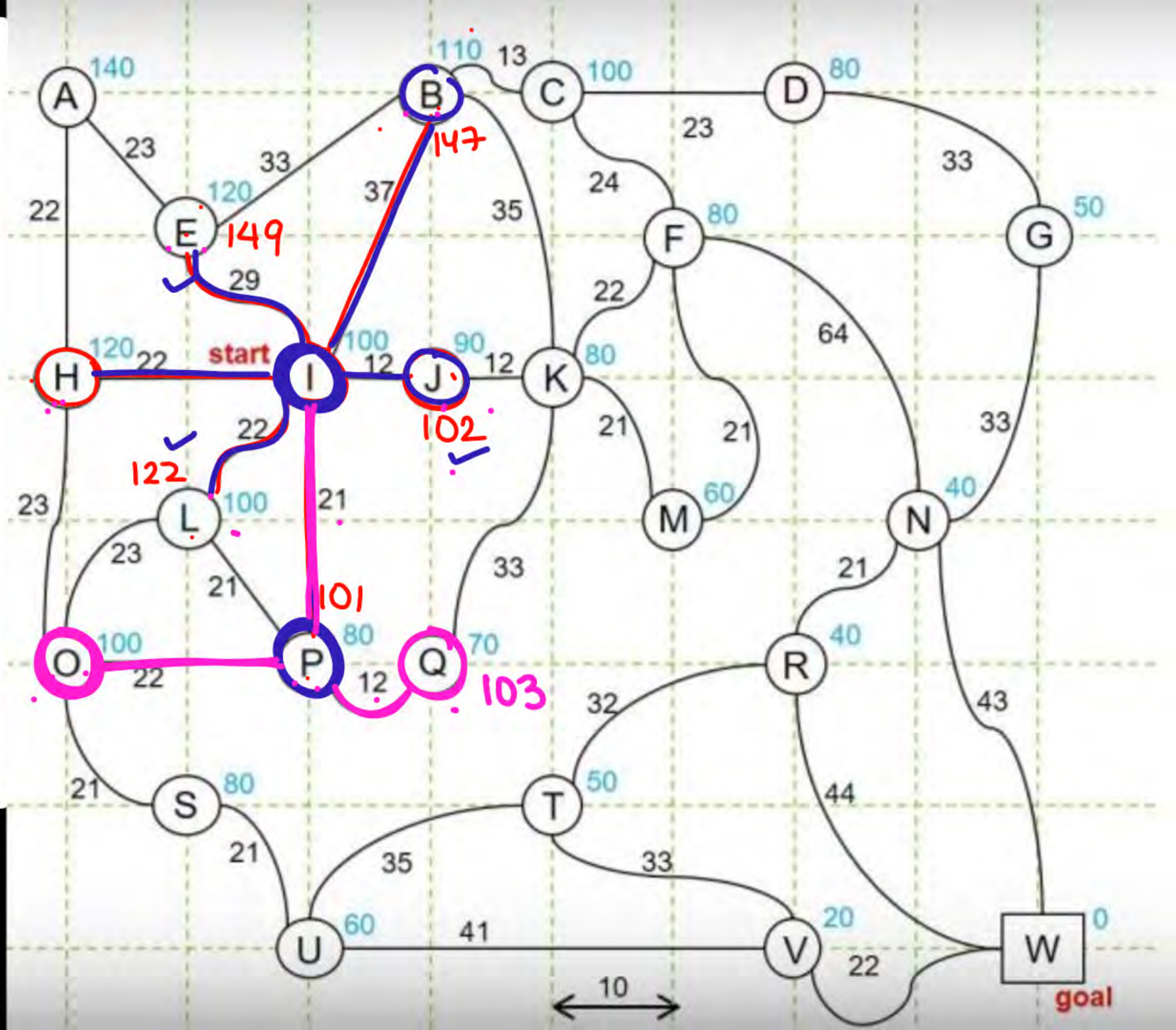
$f_{th} \Rightarrow 100$

2nd It I, P

$f_{th} \Rightarrow 101$

3rd it

$f_{th} = 102$



IDA* vs A*

- 1. Algorithm Type:
- A*: A* is a best-first search algorithm that explores nodes in the order of their estimated total path cost $f(n)=g(n)+h(n)$, where $g(n)$ is the cost to reach the node and $h(n)$ is the heuristic estimate of the cost from the node to the goal.
- IDA*: IDA* is an iterative deepening version of A*, which uses depth-first search but applies a threshold based on $f(n)$ to limit the depth of the search in each iteration. It increases the threshold progressively until the goal is found.

IDA* vs A*

- 2. Memory Usage:
- A*: Requires significant memory, as it stores all the nodes in the open and closed lists. In the worst case, this can lead to memory consumption growing exponentially with the depth of the search space.
- IDA*: Requires much less memory, similar to depth-first search. It only needs to store the current path and a small amount of additional information, making it much more memory-efficient.

IDA* vs A*

- 3. Time Complexity:
- A*: The time complexity is $O(b^d)$, where bb is the branching factor and dd is the depth of the optimal solution. It explores all possible paths but typically does so faster than uninformed search algorithms.
- IDA*: The time complexity is also $O(b^d)$, but due to its iterative nature, it may revisit nodes multiple times, making it potentially slower in practice compared to A*.

IDA* vs A*

- 4. Space Complexity:

- A*: Space complexity is $O(b^d)$, as it needs to store all explored nodes. This can be a significant limitation for large problems.
- IDA*: Space complexity is $O(b \times d)$, as it only stores the path being explored and the next threshold. This makes IDA* feasible for much larger problems than A*.

IDA* vs A*

- A* is faster in practice and is preferred when memory is sufficient.
- IDA* is more memory-efficient and suitable for large problems but may be slower due to its iterative nature.

THANK - YOU